

Agentic Simulator

Extended Code Tutorial

A Discrete Event Simulator for LLM-Based Agentic Workloads
on Edge / Cloud Infrastructure

Thesis Project - Master's Degree

May 2026

Table of Contents

- 1. Overview & Motivation
- 2. Architecture at a Glance
- 3. Package Structure
- 4. Configuration Layer (config/AgenticConfig.java)
- 5. JSON Configuration File (agentic_config.json)
- 6. Multiple Agents & Workloads
- 7. Event System (events/AgenticEvent.java)
- 8. Event Scheduler (scheduler/EventScheduler.java)
- 9. Stochastic LLM Engine (engine/LLMEngine.java)
- 10. Agent Decision Model (engine/AgentDecision.java)
- 11. Tool Pool (engine/ToolPool.java)
- 12. Infrastructure Layer (infra/InfrastructureLayer.java)
- 13. Agent Service (runtime/AgentService.java)
- 14. Orchestrator (runtime/Orchestrator.java)
- 15. Message Record (runtime/Message.java)
- 16. Trajectory Collector (metrics/TrajectoryCollector.java)
- 17. Simulation Runner (AgenticSimulationRunner.java)
- 18. Entry Point (SimulatorApplication.java)
- 19. End-to-End Flow Example (Single Workflow)
- 20. End-to-End Flow: Multiple Workloads Under Contention
- 21. Key Design Decisions
- 22. Visualization

1. Overview & Motivation

This project is a Discrete Event Simulator (DES) that models the behavior of LLM-based agentic systems running on edge/cloud infrastructure. Instead of calling real LLM APIs (which would be expensive and slow), it uses stochastic models to simulate realistic latency, cost, and decision-making behavior.

The simulator answers questions like:

- How does placing an agent on the edge vs. cloud affect end-to-end latency?
- What is the cost of running N concurrent agentic workflows per second?
- How do concurrency limits and queuing affect tail latency?
- What happens when tools fail or have high latency?
- How do multiple agents with different workloads compete for shared infrastructure?
- What is the optimal number of agent replicas for a given traffic mix?

It is the v2 evolution of a Kubernetes microservice simulator (v1). While v1 modeled traditional request/response microservices with CPU time-slicing, v2 models agentic workflows where an LLM makes decisions in a loop: generate text, call tools, or delegate.

What makes it 'agentic'?

In a traditional microservice, the request follows a fixed path (Service A -> Service B -> C). In an agentic system, the LLM DECIDES the next step at each iteration. It might:

1. Generate a final text response (workflow completes)
2. Call an external tool (search, database, API) and loop back
3. Delegate to another agent
4. Fail

This creates a variable-length, non-deterministic execution path that the simulator models using weighted random decisions and stochastic latency sampling.

2. Architecture at a Glance

The simulator follows a classic Discrete Event Simulation pattern:

1. A priority queue (min-heap) holds future events sorted by time.
2. The event loop pops the earliest event and dispatches it.
3. Processing an event may schedule new future events.
4. Time is simulated (not real) - it jumps from event to event.

This is single-threaded and deterministic given the same random seed.

Component Wiring Diagram

```
AgenticSimulationRunner
|-- loads JSON config --> Topology (N agents, M workloads)
|-- creates LLMEngine (stochastic LLM surrogate)
|-- creates ToolPool (stochastic tool surrogate)
|-- creates InfrastructureLayer (v1 CPU/queue model)
|-- creates EventScheduler (priority queue)
|-- creates TrajectoryCollector (CSV logger)
|-- creates Orchestrator (event router + workload registry)
|   |-- registers AgentService for each agent in config
|   |-- maps workflowId -> WorkloadDefinition
|
|-- generates per-workload Poisson traffic streams
|-- merges & submits all arrivals chronologically
|-- runs event loop: scheduler.run(orchestrator::process)
|-- exports trajectory CSV
```

Two-Layer Architecture

The simulator uses a two-layer latency model:

Application Layer (v2): LLM inference, tool calls, agent decisions

Infrastructure Layer (v1): CPU time-slicing, pod queuing, contention

When an agent processes a step, it first goes through the infrastructure layer (local CPU compute: context building, prompt formatting, response parsing), then dispatches the external LLM call. There is NO network cost between the two layers because they run on the same machine.

```
Total step latency = infraLatencyMs + networkOut + inferenceMs + networkBack
                    ^^^^^^^^^^^^^^^  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
                    from v1 (CPU)    from v2 (external LLM API call)
```

Event Flow

```
User Request (Poisson)
--> [SUBMIT] Orchestrator schedules AgentReceive
--> [AGENT_RECEIVE] AgentService builds context
```

```
--> submits job to InfrastructureLayer (CPU compute)
--> schedules InfraComplete (after CPU + queue latency)
--> [INFRA_COMPLETE] CPU compute done, dispatch LLM call
--> schedules LlmComplete (after network + inference)
--> [LLM_COMPLETE] Branch on decision:
    GENERATE_TEXT --> workflow SUCCESS, terminate
    CALL_TOOL     --> schedule ToolComplete
    DELEGATE      --> (reserved for multi-agent)
    FAIL         --> workflow FAIL, terminate
--> [TOOL_COMPLETE] Result feeds back as new AgentReceive
--> loop continues until text/fail/budget exhausted
```

3. Package Structure

```
com.thesis.simulator.agentic/
|-- AgenticSimulationRunner.java  (main runner, wiring, traffic gen)
|
|-- config/
|   +-- AgenticConfig.java        (all data records: Topology, Zone,
|                                   LLMPProfile, ToolProfile, InfraNode, etc.)
|
|-- engine/
|   +-- LLMEngine.java            (stochastic LLM: latency, cost, decisions)
|   +-- AgentDecision.java        (decision enum + record)
|   +-- ToolPool.java             (stochastic tool calls)
|
|-- infra/
|   +-- InfrastructureLayer.java   (v1 CPU model: time-slicing, queuing)
|   +-- InfraResult.java          (latency breakdown record)
|
|-- events/
|   +-- AgenticEvent.java         (sealed event hierarchy)
|
|-- runtime/
|   +-- Orchestrator.java         (event dispatcher + workflow submission)
|   +-- AgentService.java         (the agent loop: receive->infra->LLM->branch)
|   +-- Message.java             (token-carrying message record)
|
|-- scheduler/
|   +-- EventScheduler.java       (priority queue event loop)
|
+-- metrics/
    +-- TrajectoryCollector.java   (in-memory log, CSV export)
```

The separation is intentional:

- 'config' holds pure data (records) with no behavior.
- 'engine' holds the stochastic models (LLM, tools) - the 'math'.
- 'infra' holds the infrastructure layer (CPU contention, pod queuing) - ported from v1.
- 'events' defines the message types flowing through the system.
- 'runtime' holds the processing logic (agent loop, orchestrator).
- 'scheduler' is the DES engine itself (reusable, generic).
- 'metrics' handles observability and output.

4. Configuration Layer

File: config/AgenticConfig.java

This file contains only Java records (immutable data classes). No behavior, just structure. It defines the entire topology of a simulation experiment.

Zone

```
public record Zone(String id) {}
```

A logical placement zone like "CLOUD", "EDGE", or "FOG". Agents, LLMs, and tools are each hosted in a zone. Network latency depends on zone pairs.

NetworkLink

```
public record NetworkLink(
    String fromZone, String toZone,
    double latencyMs, double bandwidthMbps
) {}
```

A directed network link. When an agent in zone A calls an LLM in zone B, the simulator looks up the (A->B) link to add network latency. Every cross-zone call pays round-trip: A->B + B->A.

LLMProfile

```
public record LLMProfile(
    String id,
    double ttftMeanMs,           // Time To First Token (mean)
    double ttftGammaShape,      // Gamma distribution shape param
    double tpotMeanMs,          // Time Per Output Token (mean)
    double tpotGammaShape,      // Gamma distribution shape param
    int outputTokensMean,       // How many tokens per response
    int outputTokensStd,
    double pricePerInputToken,
    double pricePerOutputToken,
    String hostZone
) {}
```

Stochastic profile of an LLM model. The key insight: real LLM inference latency follows a Gamma distribution (always positive, right-skewed with occasional outliers). Total latency = TTFT + outputTokens * TPOT.

TTFT (Time To First Token) is the startup delay before streaming begins. TPOT (Time Per Output Token) is the per-token generation speed. Both are sampled from Gamma(shape, scale=mean/shape).

ToolProfile

```
public record ToolProfile(
    String id,
    double latencyMeanMs, double latencyStdMs,
```

```
    int responseTokensMean,
    double errorRate,
    String hostZone
) {}
```

Profile for external tools (search APIs, databases, MCP functions). Latency is Gaussian. Tools can fail with a configurable error rate.

InfraNode

```
public record InfraNode(
    int nodeId, String zone,
    int cores, double frequencyHz,
    double bandwidthBytesPerSec
) {}
```

Physical/virtual infrastructure node. Used by the InfrastructureLayer to model CPU time-slicing and contention. When multiple jobs run on the same node, each gets a share: $\text{speedPerJob} = (\text{cores} / \text{activeJobCount}) * \text{frequencyHz}$.

AgentDefinition

```
public record AgentDefinition(
    String id,
    String llmProfileId,           // which LLM this agent uses
    List<String> toolIds,          // which tools are available
    String hostZone,
    int maxConcurrency,           // thread pool size
    double coldStartMs,           // container startup penalty
    long instructionsPerStep,     // CPU work per agent step
    int replicas                   // number of pod replicas
) {}
```

Defines an agent service. Each agent is hosted in a zone, uses a specific LLM, has access to certain tools, and has a concurrency limit (like a Pod's thread pool). When all slots are busy, new requests queue up.

instructionsPerStep defines how much CPU work each agent step requires (context building, prompt formatting, response parsing). This is processed by the infrastructure layer before the LLM call. replicas sets how many pod instances are deployed (round-robin across nodes), matching v1's pod deployment model.

Topology & WorkloadDefinition

```
public record Topology(
    List<Zone> zones,
    List<NetworkLink> links,
    Map<String, LLMProfile> llmProfiles,
    Map<String, ToolProfile> tools,
    Map<String, AgentDefinition> agents,
    List<InfraNode> infraNodes,
    List<WorkloadDefinition> workloads
) {}

public record WorkloadDefinition(
    String name,
```

```
String entryAgent,  
double arrivalRate,  
double userMessageTokensMean,  
double userMessageTokensStdDev,  
int maxStepsPerWorkflow,  
int maxTokensPerWorkflow  
) {}
```

Topology bundles the entire experiment configuration including infrastructure nodes. WorkloadDefinition defines a distinct traffic type with its own arrival rate, token distribution, entry agent, and budget constraints (max steps and max tokens). Multiple workloads run concurrently on the same infrastructure, producing realistic resource contention between different request types.

5. JSON Configuration File

File: src/main/resources/agent_config.json

This JSON drives the entire simulation. Here is the current config with explanations:

Nodes & Zones

```
"nodes": [{
  "nodeId": 1, "zone": "CLOUD",
  "cores": 8, "frequencyHz": 3e9,
  "vramGB": 80, "bandwidthBytesPerSec": 1.25e9
}]
```

Defines physical/virtual nodes. Each node belongs to a zone (CLOUD, EDGE, etc.). These specs are used by both the zone system (for network latency) and the InfrastructureLayer (for CPU time-slicing and contention modeling).

Zone Pair Latencies

```
"zonePairLatencies": [
  { "from": "CLOUD", "to": "CLOUD", "latencyMs": 1.0 }
]
```

Network latency between zones. CLOUD->CLOUD is 1ms (intra-datacenter). For multi-zone experiments, you would add entries like EDGE->CLOUD: 50ms, CLOUD->EDGE: 50ms.

LLM Models

```
"llmModels": [{
  "modelId": "gpt-4o",
  "ttftMeanMs": 400.0, "ttftGammaShape": 4.0,
  "tpotMeanMs": 25.0, "tpotGammaShape": 10.0,
  "outputTokensMean": 150.0, "outputTokensStdDev": 50.0,
  "costPerInputToken": 0.0000025,
  "costPerOutputToken": 0.00001,
  "hostZone": "CLOUD",
  "decisionWeights": { "generate_text": 0.4, "call_tool": 0.6 }
}]
```

This defines a GPT-4o-like model with:

- TTFT ~ Gamma(shape=4, mean=400ms) -> typical range: 100-800ms
- TPOT ~ Gamma(shape=10, mean=25ms) -> typical range: 15-40ms per token
- Output length ~ Gaussian(mean=150, std=50) tokens
- Cost: \$2.50/M input tokens, \$10/M output tokens
- Decision weights: 60% chance to call a tool, 40% to generate final text

Tools

```
"tools": [{
  "toolId": "search-tool",
```

```
"latencyMeanMs": 200.0, "latencyStdDevMs": 50.0,
"resultTokensMean": 300,
"errorRate": 0.0,
"hostZone": "CLOUD"
}]
```

A search tool with ~200ms latency returning ~300 tokens. 0% error rate.

Agent Services

The agentServices array accepts any number of agents. Each agent is an independent service with its own concurrency pool, LLM model, tools, and replica count. Adding a new agent is purely a config change - no code modification required.

```
"agentServices": [
  {
    "serviceId": "agent-chat",
    "nodeId": 1, "modelId": "gpt-4o",
    "maxConcurrency": 10, "hostZone": "CLOUD",
    "instructionsPerStep": 50000000,
    "replicas": 2
  },
  {
    "serviceId": "agent-tool",
    "nodeId": 1, "modelId": "gpt-4o",
    "maxConcurrency": 5, "hostZone": "CLOUD",
    "instructionsPerStep": 50000000,
    "replicas": 3
  }
]
```

Each agent has infrastructure properties:

- instructionsPerStep: 50M instructions of CPU work per step (context building, prompt formatting, response parsing). On an 8-core 3GHz node with no contention, this takes ~2ms. Under heavy load with CPU sharing, it takes longer.
- replicas: number of pod instances deployed round-robin across infrastructure nodes.
- maxConcurrency: thread pool size per replica.

Agents run independently but share physical nodes. agent-chat (2x10 = 20 slots) and agent-tool (3x5 = 15 slots) have separate queues, but their CPU work competes on the same hardware.

Simulation Parameters & Workloads

```
"simulation": {
  "durationSeconds": 100.0,
  "seed": 42
}

"workloads": [
  {
    "name": "chat-light",
    "entryAgent": "agent-chat",
    "arrivalRate": 3.0,
    "userMessageTokensMean": 50.0,
    "userMessageTokensStdDev": 20.0,
    "maxStepsPerWorkflow": 5,

```

```
    "maxTokensPerWorkflow": 4000
  },
  {
    "name": "tool-heavy",
    "entryAgent": "agent-tool",
    "arrivalRate": 2.0,
    "userMessageTokensMean": 200.0,
    "userMessageTokensStdDev": 60.0,
    "maxStepsPerWorkflow": 15,
    "maxTokensPerWorkflow": 50000
  }
]
```

The simulation block defines global parameters (duration, seed). The workloads array defines one or more concurrent traffic types. Each workload has its own Poisson arrival rate, token distribution, entry agent, and budget constraints (max steps and max tokens). The entryAgent field routes each workload to a specific agent.

In this example, 'chat-light' routes to agent-chat (3.0/s, ~50 tokens, max 5 steps) while 'tool-heavy' routes to agent-tool (2.0/s, ~200 tokens, max 15 steps). Both agents share the same physical node, creating CPU contention while maintaining separate concurrency pools. You can add as many workloads and agents as needed.

6. Multiple Agents & Workloads

A key feature of v2 is the ability to define an arbitrary number of agents and workloads, all running simultaneously on the same infrastructure. Both the `agentServices` and `workloads` arrays in the JSON config accept any number of entries - there is no hard-coded limit. This models real-world scenarios where different request types are handled by different specialized agents competing for shared infrastructure resources like CPU, thread pools, and LLM capacity.

Multiple Agents

Each entry in the `agentServices` array defines a separate agent with its own:

- LLM model (`modelId`)
- Concurrency limit (`maxConcurrency`) and replica count
- Available tools
- Host zone and CPU cost per step (`instructionsPerStep`)

Agents operate independently: each has its own concurrency slots and request queue. A workflow routed to `agent-chat` does not consume concurrency slots on `agent-tool`. However, all agents share the same physical infrastructure nodes, so CPU contention still occurs when multiple agents run on the same node.

```
"agentServices": [
  {
    "serviceId": "agent-chat",
    "nodeId": 1, "modelId": "gpt-4o",
    "maxConcurrency": 10, "replicas": 2,
    "instructionsPerStep": 50000000
  },
  {
    "serviceId": "agent-tool",
    "nodeId": 1, "modelId": "gpt-4o",
    "maxConcurrency": 5, "replicas": 3,
    "instructionsPerStep": 50000000
  }
]
```

In this example, `agent-chat` has 2 replicas with 10 slots each (20 total concurrent workflows), while `agent-tool` has 3 replicas with 5 slots each (15 total). They can use different LLM models, different tools, and even run in different zones. The simulator handles any combination automatically.

Multiple Workloads

Each workload defined in the JSON config generates its own independent Poisson arrival stream. At startup, the simulation runner:

1. Iterates over each workload definition
2. Generates arrivals using exponential inter-arrival times ($\text{rate} = \text{workload.arrivalRate}$)
3. Tags each workflow with the workload name (e.g., `'wf-chat-light-0001'`)
4. Merges all arrivals into a single chronological stream
5. Submits them all to the same event queue

Each workload specifies an `entryAgent`, which determines which agent handles its workflows. Different workloads can target different agents, or multiple workloads can target the same agent.

```
"workloads": [
  { "name": "chat-light", "entryAgent": "agent-chat", "arrivalRate": 3.0, ... },
  { "name": "tool-heavy", "entryAgent": "agent-tool", "arrivalRate": 2.0, ... },
  { "name": "batch-job", "entryAgent": "agent-tool", "arrivalRate": 0.5, ... }
]
```

In this example, chat-light flows to agent-chat, while both tool-heavy and batch-job flow to agent-tool. The two workloads on agent-tool compete for its 15 concurrency slots, while chat-light runs independently on agent-chat's 20 slots. All three workloads still share the same physical CPU node.

Routing: Workload to Agent Mapping

The `entryAgent` field in each workload definition is the only link between workloads and agents. This is purely a config-level routing decision - no code changes are needed to add new agents or workloads. The Orchestrator reads the `entryAgent` field and dispatches the initial `AgentReceive` event to the correct `AgentService`.

This means you can model diverse scenarios:

- All workloads on one agent (shared capacity, maximum contention)
- Each workload on its own agent (isolated capacity, no cross-workload queuing)
- Mixed routing (some workloads share agents, others are isolated)
- Agents in different zones (edge agent for latency-sensitive, cloud for heavy)

Per-Workload Budget

Each workload has its own `maxStepsPerWorkflow` and `maxTokensPerWorkflow`. These limits are stored in the Orchestrator's workload registry and passed to the `AgentService` when a `WorkflowContext` is created. This means a 'chat-light' workflow with `maxSteps=5` will terminate with `BUDGET_EXHAUSTED` after 5 agent steps, while a 'tool-heavy' workflow on the same agent can continue for up to 15 steps.

Scalability

There is no limit on the number of agents, workloads, nodes, LLM models, or tools you can define. The simulator parses all arrays from the JSON config and wires them together automatically. You can model scenarios ranging from a single agent with one workload to dozens of specialized agents across multiple zones handling hundreds of distinct workload types. Adding a new agent or workload is always a config change - never a code change.

Metrics Breakdown

The trajectory CSV includes a 'workload' column in `SUBMIT` and `COMPLETE` events, and an `entity_id` column identifying which agent processed each step. This allows post-simulation analysis to break down latency, cost, success rate, and step count per workload type and per agent. The `printSummary()` method reports both aggregate and per-workload statistics.

Example Output

```
=== Agentic Simulation Summary ===
--- ALL ---
  Completed:    495 | Successful: 473 (96%)
  Avg latency:  9842.1 ms (9.842 s)
  Max latency:  43931.2 ms (43.931 s)
  Total cost:   $3.0814 | Total steps: 2306
--- chat-light (agent-chat) ---
  Completed:    296 | Successful: 274 (93%)
  Avg latency:  9890.3 ms (9.890 s)
  Max latency:  28762.5 ms (28.763 s)
  Total cost:   $1.6506 | Total steps: 1191
--- tool-heavy (agent-tool) ---
  Completed:    199 | Successful: 199 (100%)
  Avg latency:  9790.8 ms (9.791 s)
  Max latency:  43931.2 ms (43.931 s)
  Total cost:   $1.4308 | Total steps: 1115
```

7. Event System

File: events/AgenticEvent.java

This uses Java's sealed interface + records for type-safe, exhaustive event types. Every event has a `timeMs()` and is `Comparable` (sorted by time in the priority queue).

Event Types

TaskSubmit

Reserved placeholder for queued submissions. Not used in the current flow.

AgentReceive

```
record AgentReceive(double timeMs, String workflowId,
                   String agentId, Message message)
```

A message has arrived at an agent and is ready for processing. This is the entry point for every agent step. Carries the input tokens via `Message`.

InfraComplete

```
record InfraComplete(double timeMs, String workflowId,
                    String agentId, String infraJobId,
                    AgentDecision decision, double cost,
                    int outputTokens, double networkOut,
                    double inferenceMs, double networkBack)
```

The infrastructure compute (CPU work) has completed. The agent can now dispatch the external LLM call. This event carries forward all the pre-sampled LLM values (decision, cost, tokens, latency) so the LLM call can be scheduled immediately. On firing, the infrastructure resources are released (CPU slot freed, next queued job dequeued).

LlmComplete

```
record LlmComplete(double timeMs, String workflowId,
                  String agentId, AgentDecision decision,
                  double costAccrued, int outputTokens)
```

The LLM has finished generating. The decision field tells the agent what to do next: generate text (finish), call a tool (loop), delegate, or fail.

ToolComplete

```
record ToolComplete(double timeMs, String workflowId,
                   String agentId, String toolId,
                   int responseTokens, boolean errored)
```

A tool call has returned. If `errored=true`, the workflow terminates. Otherwise, the response tokens feed back as a new `AgentReceive`.

WorkflowComplete

```
record WorkflowComplete(double timeMs, String workflowId,
```

```
String reason, double totalLatencyMs,  
double totalCostUsd, int totalSteps)
```

Terminal event. Reason can be: SUCCESS, BUDGET_EXHAUSTED, TOOL_FAILURE, or LLM_FAIL.

8. Event Scheduler

File: scheduler/EventScheduler.java

The heart of any DES. This is intentionally minimal - just a priority queue wrapper.

```
public class EventScheduler {
    private final PriorityQueue<AgenticEvent> queue = new PriorityQueue<>();
    private double now = 0.0;

    public void schedule(AgenticEvent ev) {
        // Cannot schedule events in the past
        if (ev.timeMs() < now) throw ...;
        queue.offer(ev);
    }

    public void run(Consumer<AgenticEvent> handler) {
        while (!queue.isEmpty()) {
            AgenticEvent ev = queue.poll(); // earliest event
            now = ev.timeMs();              // advance clock
            handler.accept(ev);              // process it
        }
    }
}
```

Key points:

- PriorityQueue sorts events by timeMs() (via Comparable).
- The run() loop is the main simulation loop - it processes ALL events.
- Processing one event (e.g., AgentReceive) typically schedules new future events (e.g., LlmComplete at time now + latency). This is how the simulation progresses.
- Single-threaded: no real concurrency, just simulated time.

9. Stochastic LLM Engine

File: engine/LLMEngine.java

This is the mathematical core. It does NOT call any real LLM API. Instead, it samples from probability distributions that match real-world LLM behavior.

Latency Model

```
latency = TTFT + outputTokens * TPOT

TTFT ~ Gamma(shape, scale = mean/shape)
TPOT ~ Gamma(shape, scale = mean/shape)
```

Why Gamma distribution?

- Always positive (latency cannot be negative)
- Right-skewed (occasional high-latency outliers, matching real LLM behavior)
- Two-parameter: shape controls the 'spread', mean sets the center
- When shape is large (e.g., 10), Gamma approaches Gaussian
- When shape is small (e.g., 2), there's a long right tail

The implementation uses Marsaglia & Tsang's (2000) rejection sampling method, which is the standard efficient algorithm for Gamma sampling.

Decision Model

```
public AgentDecision decide(int inputTokens, List<String> availableTools) {
    return decisionPolicy.apply(inputTokens, availableTools);
}
```

The decision policy is a pluggable BiFunction injected via the constructor. The current implementation is a weighted random chooser:

- Roll a random number [0, 1)
- Walk through the cumulative weights (e.g., call_tool=0.6, generate_text=0.4)
- Return the first action whose cumulative weight exceeds the roll

This means with the default config: 60% of LLM steps call a tool, 40% finish.

Cost Model

```
cost = inputTokens * pricePerInputToken + outputTokens * pricePerOutputToken
```

Straightforward token-based pricing. Accumulated per workflow.

10. Agent Decision Model

File: engine/AgentDecision.java

```
public record AgentDecision(Kind kind, String targetId, int outputTokens) {
    public enum Kind {
        GENERATE_TEXT, // Final answer -> workflow completes
        CALL_TOOL,      // Invoke a tool -> loop back
        DELEGATE,        // Hand off to another agent
        FAIL             // LLM failure
    }
}
```

Simple value object representing what the LLM decided to do:

- GENERATE_TEXT: The agent produces a final answer. targetId is null.
- CALL_TOOL: targetId = the tool ID (e.g., 'search-tool').
- DELEGATE: targetId = another agent's ID. (Not yet implemented in v2.)
- FAIL: Simulates LLM failure/hallucination.

Factory methods (text(), tool(), delegate(), fail()) create instances cleanly.

11. Tool Pool

File: engine/ToolPool.java

Stochastic surrogate for external tool calls. Much simpler than LLMEngine.

```
sampleLatencyMs(toolId):
    latency ~ Gaussian(mean, stdDev), clamped >= 0

sampleError(toolId):
    return random() < errorRate

sampleResponseTokens(toolId):
    return profile.responseTokensMean (fixed, not random)
```

Tool latency uses Gaussian (can go near zero, unlike LLM which uses Gamma). Error sampling is a simple Bernoulli trial. Response tokens are currently fixed (could be made stochastic in a future version).

12. Infrastructure Layer

File: infra/InfrastructureLayer.java

This class bridges v1 (Kubernetes simulator) and v2 (agentic simulator). It models the physical infrastructure that agents run on: CPU time-slicing, pod queuing, and contention effects. The math is ported directly from v1's Simulation.java.

Why Two Layers?

In v2, agents are microservices running on physical hardware. When an agent processes a step (builds context, formats prompt, parses response), that work requires CPU time on a real node. If multiple workflows hit the same node simultaneously, they share CPU and each gets slower.

Previously, v2 only modeled application-level latency (LLM inference, tool calls). Now the infrastructure layer adds the hardware cost on top, giving realistic contention effects under load.

State (persists across calls)

```
Map<String, List<Pod>> agentToPods      // pods per agent
Map<Integer, List<ActiveJob>> nodeActiveJobs // CPU jobs per node
Map<Integer, InfraNode> nodeMap          // node specs
Map<String, ActiveJob> jobIndex          // lookup by job ID
```

Inner Classes

Pod

Same concept as v1's Pod: a container replica with a thread pool. Has activeRequests counter, maxConcurrency limit, and a request queue.

ActiveJob

Tracks a running job on a CPU: remaining instructions, current speed (recalculated when jobs arrive/depart), and the pod it belongs to.

submitJob(agentId, jobId, nowMs, instructions)

Called when an agent step begins. Returns estimated infrastructure latency:

1. Pick pod via round-robin load balancing
2. If pod at capacity: estimate queue wait from earliest-finishing job
3. updateProgress(): settle all running jobs to current time
4. Add new ActiveJob to the node
5. rescheduleNode(): recalculate speeds for all jobs on the node
 $\text{speedPerJob} = (\text{cores} / \text{totalActiveJobs}) * \text{frequencyHz}$
6. $\text{computeTimeMs} = (\text{instructions} / \text{speedPerJob}) * 1000$
7. Return `InfraResult(queueWaitMs + computeTimeMs)`

releaseJob(jobId, nowMs)

Called when infrastructure compute completes (InfraComplete event fires):

1. Update progress on all node jobs
2. Remove completed job from active list
3. Free pod thread (activeRequests--)
4. Dequeue next waiting job if any
5. Recalculate speeds (remaining jobs speed up)

Contention Example

Node: 8 cores, 3GHz. Agent step: 50M instructions.

1 concurrent job: speed = $8 * 3\text{GHz} = 24\text{GHz}$ -> $50\text{M}/24\text{G} = 2.1\text{ms}$

5 concurrent jobs: speed = $1.6 * 3\text{GHz} = 4.8\text{GHz}$ -> $50\text{M}/4.8\text{G} = 10.4\text{ms}$

10 concurrent jobs: speed = $0.8 * 3\text{GHz} = 2.4\text{GHz}$ -> $50\text{M}/2.4\text{G} = 20.8\text{ms}$

Under heavy load, infrastructure latency grows 10x. This is the whole point of integrating v1's model.

InfraResult

```
public record InfraResult(
    double infraLatencyMs, // total: queue + compute
    double queueWaitMs,   // time spent waiting for pod capacity
    double computeMs      // time spent on CPU
) {}
```

13. Agent Service - The Agent Loop

File: runtime/AgentService.java

This is the most complex class - it implements the full agentic processing loop. Think of it as a simulated microservice container running an LLM agent.

Concurrency Model

```
private int activeRequests = 0;
private final Queue<AgenticEvent.AgentReceive> waitingQueue = new LinkedList<>();
```

Each agent has maxConcurrency slots (like a thread pool). When all slots are busy, new requests enter a FIFO queue. When a workflow completes, it frees a slot and the next queued request is dequeued. This mirrors real container behavior (Tomcat/Jetty thread pools in Kubernetes Pods).

Per-Workflow State: WorkflowContext

```
private static class WorkflowContext {
    final String workflowId;
    final double startedAtMs;
    int stepIndex = 0;
    int accumulatedInputTokens = 0;
    int accumulatedOutputTokens = 0;
    double totalCostUsd = 0.0;
}
```

Each workflow gets its own context tracking: step count, accumulated tokens, and cost. This is keyed by workflowId in a HashMap. Context is created on first receive and destroyed on termination.

Step-by-Step Processing

Step 1: onReceive(AgentReceive)

Entry point. Checks concurrency:

- If at capacity: queue the request, log QUEUE_ENTER, return.
- Otherwise: increment activeRequests, proceed to processReceive().

Step 2: processReceive()

- Get or create WorkflowContext for this workflowId.
- Increment step counter, accumulate input tokens.
- Budget check: if steps > maxSteps or tokens > maxTokens, terminate.
- Look up the LLM profile (from agent's llmProfileId).
- Calculate network latency: agent zone -> LLM zone -> agent zone.
- Sample output tokens, decision, inference latency, and cost from LLMEngine.
- Submit job to InfrastructureLayer (CPU compute for context building).
- Schedule InfraComplete at time: now + infraLatencyMs.
(carries forward all LLM values for the next step)

Step 3: onInfraComplete(InfraComplete)

Infrastructure CPU compute has finished:

- Release infrastructure resources (infraLayer.releaseJob).
- Now schedule the external LLM call.
- Schedule LlmComplete at time: now + networkOut + inference + networkBack.

Step 4: onLlmComplete(LlmComplete)

The LLM has responded. Branch on the decision:

- GENERATE_TEXT -> terminate with SUCCESS
- CALL_TOOL -> dispatchTool()
- DELEGATE -> terminate with DELEGATE_NOT_IMPLEMENTED
- FAIL -> terminate with LLM_FAIL

Step 5: dispatchTool()

- Look up tool profile for network zone.
- Calculate network latency: agent zone -> tool zone -> agent zone.
- Sample tool latency and error flag.
- Schedule ToolComplete at time: now + networkOut + toolMs + networkBack.

Step 6: onToolComplete(ToolComplete)

- If errored: terminate with TOOL_FAILURE.
- Otherwise: create a follow-up Message with the tool's response tokens.
- Call processReceive() directly (no re-check concurrency - this workflow already holds a slot). This creates the LOOP back to Step 2.

Termination: terminate()

- Schedule WorkflowComplete event with total latency, cost, and step count.
- Remove WorkflowContext.
- Decrement activeRequests (free concurrency slot).
- Call tryDequeue() to process waiting requests.

14. Orchestrator

File: runtime/Orchestrator.java

Simple event router and workflow entry point. Two responsibilities:

1. submit(workflowId, promptTokens, now)

Creates the initial Message (from 'USER' to the entry agent) and schedules the first AgentReceive event. This is how workflows enter the system.

2. process(AgenticEvent)

Central dispatch method wired into the scheduler's main loop. Uses Java's pattern matching (instanceof) to route each event type to the correct AgentService method:

```
AgentReceive    -> agents.get(agentId).onReceive()
InfraComplete   -> agents.get(agentId).onInfraComplete()
LlmComplete     -> agents.get(agentId).onLlmComplete()
ToolComplete    -> agents.get(agentId).onToolComplete()
WorkflowComplete -> log final metrics to TrajectoryCollector
```

15. Message Record

File: runtime/Message.java

```
public record Message(
    long id,           // auto-incrementing
    String fromAgent,  // sender ('USER', 'search-tool', etc.)
    String toAgent,    // receiver (agent ID)
    int inputTokens,   // token count (not bytes)
    Object payload,    // type-erased content
    double createdAtMs // timestamp
) {}
```

The data unit exchanged between entities. Key difference from v1's ServiceCall: it carries TOKEN counts, not byte counts. This is the fundamental unit of agentic workloads - everything is measured in tokens.

16. Trajectory Collector

File: metrics/TrajectoryCollector.java

In-memory event log that writes to CSV. Every significant event in the simulation is recorded with a flexible key-value payload.

```
void log(workflowId, entityId, eventType, timeMs, Map<String, Object> payload)

// Creates a Row(workflowId, entityId, eventType, timeMs, payload)
// Payload keys are dynamic (different per event type)
```

The writeCsv() method discovers ALL payload keys across ALL rows and writes them as columns. This means the CSV schema is self-adapting: adding new payload fields in the code automatically creates new CSV columns.

Example output columns:

time_ms, workflow_id, entity_id, event_type, step, input_tokens, decision, cost_usd, network_out_ms, inference_ms, ...

17. Simulation Runner

File: AgenticSimulationRunner.java

The main orchestration class. It wires everything together and runs the simulation. Here is what run(configPath) does, step by step:

Phase 1: Load Configuration

Reads the JSON config file using Jackson. Parses it into a Topology object containing all zones, links, LLM profiles, tool profiles, and agent definitions.

Phase 2: Build Decision Policy

Creates the weighted random decision function from the config's decisionWeights. This is a lambda/BiFunction that, given (inputTokens, availableTools), rolls a random number and picks call_tool, generate_text, delegate, or fail based on cumulative probabilities.

Phase 3: Create Components

```
LLMEngine llm = new LLMEngine(new Random(seed), decisionPolicy);
ToolPool tools = new ToolPool(topology.tools(), new Random(seed+1));
InfrastructureLayer infraLayer = new InfrastructureLayer(
    topology.infraNodes(), topology.agents());
EventScheduler scheduler = new EventScheduler();
TrajectoryCollector trace = new TrajectoryCollector();
Orchestrator orchestrator = new Orchestrator(topology, scheduler, trace);

// Register all agents
for each agent in topology:
    AgentService agent = new AgentService(
        def, topology, llm, tools, scheduler, trace, infraLayer);
    orchestrator.registerAgent(agentId, agent);
```

Phase 4: Generate Per-Workload Traffic (Poisson Process)

```
for each workload in topology.workloads():
    while (currentArrivalMs <= totalDuration):
        promptTokens = Gaussian(workload.tokensMean, workload.tokensStdDev)
        allArrivals.add(time, 'wf-{workload}-NNNN', workload, promptTokens)
        gap = -ln(U) / workload.arrivalRate // Exponential inter-arrival
        currentArrivalMs += gap

sort allArrivals by time
for each arrival: orchestrator.submit(workflowId, workload, promptTokens, time)
```

Each workload generates its own independent Poisson stream. All arrivals are merged chronologically and submitted to the same event queue. This means different workload types compete for the same infrastructure resources. For example, with chat-light (0.3/s) and tool-heavy (0.2/s) over 100s, you get ~30 + ~20 = ~50 total workflows interleaved in time.

Phase 5: Run Event Loop

```
scheduler.run(orchestrator::process);
```

One line, but this IS the entire simulation. The scheduler drains its priority queue, calling `orchestrator.process()` for each event. Processing events schedules new events. The loop runs until no more events exist.

Phase 6: Export & Summary

Writes trajectory CSV and prints summary statistics: completed workflows, success rate, average/max latency, total cost, total steps.

18. Entry Point

File: SimulatorApplication.java

Spring Boot application with two modes:

- Default (no args): runs v1 Kubernetes microservice simulation.
- With "agentic" argument: runs v2 agentic simulation.

The agentic path simply creates an AgenticSimulationRunner and calls run().

```
if (args contains "agentic") {  
    new AgenticSimulationRunner().run("agentic_config.json");  
} else {  
    // v1 Kubernetes simulation  
}
```

19. End-to-End Flow Example (Single Workflow)

Let's trace a single workflow through the entire system with the default config: 1 agent (agent-1) in CLOUD with 2 replicas, 1 LLM (gpt-4o) in CLOUD, 1 tool (search-tool) in CLOUD, intra-cloud latency = 1ms, node: 8 cores @ 3GHz, agent: 50M instructions/step.

t=0.000ms: Workflow Submitted

Poisson traffic generator creates workflow 'wf-0001' with 50 prompt tokens. Orchestrator creates Message(USER -> agent-1, 50 tokens) and schedules AgentReceive at t=0.

t=0.000ms: Agent Receives Message (Step 1)

AgentService.onReceive() checks concurrency: 0 active < 10 max. Passes.

processReceive():

- Creates WorkflowContext(startedAt=0)
- stepIndex = 1, accumulatedInputTokens = 50
- Budget check: $1 \leq 10$ steps, $50 \leq 10000$ tokens. OK.
- Samples outputTokens ~ Gaussian(150, 50) = e.g., 142
- Samples decision: $\text{roll}=0.35 < 0.6$ (call_tool weight) -> CALL_TOOL('search-tool')
- Samples TTFT ~ Gamma(4, 400) = e.g., 380ms
- Samples TPOT ~ Gamma(10, 25) = e.g., 24ms
- Inference = $380 + 142 * 24 = 3788\text{ms}$
- Cost = $50 * \$2.5\text{e-}6 + 142 * \$10\text{e-}6 = \$0.001545$

t=0.000ms: Infrastructure Compute (INFRA_SUBMIT)

Before dispatching the LLM call, the agent does local CPU work (context building, prompt formatting). This goes through the infrastructure layer:

- infraLayer.submitJob('agent-1', jobId, 0ms, 50M instructions)
- Node: 8 cores, 3GHz, 0 other active jobs
- speedPerJob = $(8/1) * 3\text{GHz} = 24\text{GHz}$
- computeMs = $(50\text{M} / 24\text{G}) * 1000 = 2.08\text{ms}$
- queueWaitMs = 0 (pod has capacity)
- Schedules InfraComplete at $t = 0 + 2.08 = 2.08\text{ms}$

t=2.08ms: Infrastructure Complete (INFRA_COMPLETE)

CPU compute done. infraLayer.releaseJob() frees the CPU slot.

Now the agent dispatches the external LLM call:

- Network: CLOUD->CLOUD = 1ms out + 1ms back = 2ms
- Schedules LlmComplete at $t = 2.08 + 1 + 3788 + 1 = 3792.08\text{ms}$

t=3792ms: LLM Completes

Decision was CALL_TOOL('search-tool'). Agent dispatches tool call (external API):

- Tool profile: search-tool in CLOUD
- Network: 1ms + 1ms = 2ms
- Tool latency $\sim$ Gaussian(200, 50) = e.g., 185ms
- Error check: random() = 0.72 $\geq$ 0.0 (errorRate) -> no error
- Response tokens = 300
- Schedules ToolComplete at $t = 3792 + 1 + 185 + 1 = 3979$ ms

Note: No infrastructure cost for tool calls - tools are external APIs.

t=3979ms: Tool Completes

Tool succeeded. Creates follow-up Message(search-tool -> agent-1, 300 tokens). Calls processReceive() directly (reuses concurrency slot).

t=3979ms: Agent Receives Tool Result (Step 2)

The tool result arrives back at the agent. processReceive() is called again, which triggers a NEW LLM call (LLM call #2) so the LLM can evaluate the tool result and decide what to do next. First, the LLM parameters are sampled:

- stepIndex = 2, accumulatedInputTokens = 50 + 300 = 350
- Budget check: $2 \leq 10$, $350 \leq 10000$. OK.
- Samples outputTokens $\sim$ Gaussian(150, 50) = e.g., 168
- Samples decision: roll=0.71 $\geq$ 0.6 -> GENERATE_TEXT
- Samples TTFT $\sim$ Gamma(4, 400) = e.g., 390ms
- Samples TPOT $\sim$ Gamma(10, 25) = e.g., 24ms
- Inference = $390 + 168 * 24 = 4422$ ms
- Cost = $350 * \$2.5e-6 + 168 * \$10e-6 = \$0.002555$

These values are carried forward through the infrastructure step and used when the actual LLM call is dispatched after infra compute completes.

t=3979ms: Infrastructure Compute #2 (INFRA_SUBMIT)

Same as Step 1 - agent does local CPU work before the LLM call:

- infraLayer.submitJob('agent-1', jobId, 3979ms, 50M instructions)
- Node may have other active jobs now (depends on concurrent workflows)
- If 1 other job active: speed = (8/2) * 3GHz = 12GHz
-> computeMs = $(50M / 12G) * 1000 = 4.17$ ms
- If unloaded: computeMs = 2.08ms (same as Step 1)
- Schedules InfraComplete at $t = 3979 + 2.08 = 3981.08$ ms

t=3981ms: Infrastructure Complete -> LLM Call #2 Dispatched

CPU compute done. Release infra resources. Dispatch LLM call:

- Schedules LlmComplete at $t = 3981 + 1 + 4422 + 1 = 8405$ ms

t=8405ms: LLM Completes -> SUCCESS

The LLM's decision is GENERATE_TEXT. Workflow terminates:

- Total latency: 8405 - 0 = 8405ms (8.4 seconds)
- Total cost: \$0.001545 + \$0.002555 = \$0.0041
- Total steps: 2
- Reason: SUCCESS

If the decision had been CALL_TOOL instead, the agent would have dispatched another tool call and the loop would continue (Step 3, Step 4, etc.) until the LLM finally decides GENERATE_TEXT or the budget is exhausted.

Concurrency slot freed. Next queued workflow (if any) is dequeued.

t=8405ms: Orchestrator Logs COMPLETE

TrajectoryCollector records the final row with all metrics. This workflow generated ~14 trace events total:

1. SUBMIT - workflow enters the system
2. AGENT_RECEIVE - agent gets user message
3. INFRA_SUBMIT - infra compute #1 starts (CPU work)
4. INFRA_COMPLETE - infra compute #1 done
5. LLM_DISPATCH - LLM call #1 sent (agent -> LLM)
6. LLM_COMPLETE - LLM call #1 returns: CALL_TOOL
7. TOOL_DISPATCH - tool call sent (agent -> tool)
8. TOOL_RETURN - tool result comes back
9. AGENT_RECEIVE - agent receives tool result
10. INFRA_SUBMIT - infra compute #2 starts
11. INFRA_COMPLETE - infra compute #2 done
12. LLM_DISPATCH - LLM call #2 sent (agent -> LLM)
13. LLM_COMPLETE - LLM call #2 returns: GENERATE_TEXT
14. COMPLETE - workflow finished

Notice the two-layer pattern: every agent step goes through INFRA first (CPU compute), then LLM (external API). The infra latency is small (~2ms) when unloaded, but grows under contention as concurrent workflows share CPU resources.

20. End-to-End Flow: Multiple Workloads Under Contention

Chapter 19 traced a single workflow through the system in isolation. This chapter shows how multiple workloads interact and compete for shared infrastructure, which is the key value of the multi-workload feature.

Setup

Configuration: 1 node (8 cores, 3GHz), 1 agent (agent-1, maxConcurrency=10, 2 replicas), 1 LLM (gpt-4o), 1 tool (search-tool). Two concurrent workloads:

```
chat-light:  arrivalRate=3.0/s | tokens~N(50,20) | maxSteps=5 | maxTokens=4000
tool-heavy:  arrivalRate=2.0/s | tokens~N(200,60) | maxSteps=15 | maxTokens=50000
```

Over 100 seconds, this produces roughly 300 chat-light + 200 tool-heavy = 500 total workflows, all competing for the same 2 agent replicas with 10 concurrency slots each.

Phase 1: Traffic Generation & Merging

At startup, the simulation runner generates two independent Poisson arrival streams:

```
chat-light stream:  t=0.12, t=0.45, t=0.89, t=1.02, t=1.31, ...
tool-heavy stream:  t=0.38, t=0.97, t=1.54, t=2.11, ...

Merged (sorted):    t=0.12(chat), t=0.38(tool), t=0.45(chat), t=0.89(chat),
                    t=0.97(tool), t=1.02(chat), t=1.31(chat), t=1.54(tool), ...
```

Each arrival is tagged with its workload name (e.g., 'wf-chat-light-0001', 'wf-tool-heavy-0042'). The Orchestrator stores a workload registry mapping each workflowId to its WorkloadDefinition, so budget limits are enforced per-workload.

Phase 2: Early Simulation (Low Contention)

In the first few seconds, the system is relatively unloaded:

```
t=0.12ms:  wf-chat-light-0001 arrives -> agent slot 1/20 used
           infra: 1 active job, speed = 8*3GHz = 24GHz -> compute = 2.08ms

t=0.38ms:  wf-tool-heavy-0001 arrives -> agent slot 2/20 used
           infra: 2 active jobs, speed = 4*3GHz = 12GHz -> compute = 4.17ms

t=0.45ms:  wf-chat-light-0002 arrives -> agent slot 3/20 used
           infra: 3 active jobs, speed = 2.67*3GHz = 8GHz -> compute = 6.25ms
```

Notice how each new concurrent job reduces the CPU speed for ALL jobs on the same node. Even at low load, the infrastructure layer is already showing contention effects. The compute time per step grows from 2ms to 6ms with just 3 concurrent workflows.

Phase 3: Steady State (High Contention)

With $3.0 + 2.0 = 5.0$ arrivals per second and each workflow taking multiple seconds (LLM inference alone is ~4 seconds per step), the system quickly builds up concurrent workflows. After ~10 seconds:

```
Active workflows:      ~15-25 concurrent
Agent concurrency:     near or at 20/20 (10 per replica x 2 replicas)
Queue depth:           workflows waiting for concurrency slots
CPU contention:         15+ active jobs sharing 8 cores
  -> speed per job = (8/15)*3GHz = 1.6GHz
  -> compute per step = 50M/1.6G*1000 = 31.25ms (15x slower than unloaded)
```

At this point, three bottlenecks are visible:

1. CPU contention: infrastructure compute grows from ~2ms to ~30ms+ per step
2. Agent queuing: workflows wait for a free concurrency slot
3. Resource competition: chat-light and tool-heavy workflows steal capacity from each other

Phase 4: Per-Workload Budget Enforcement

The two workloads have different budget constraints, which leads to different termination patterns:

```
chat-light workflow (maxSteps=5, maxTokens=4000):
  Step 1: 50 input tokens -> CALL_TOOL (60% probability)
  Step 2: +300 tool tokens = 350 total -> CALL_TOOL
  Step 3: +300 tool tokens = 650 total -> CALL_TOOL
  Step 4: +300 tool tokens = 950 total -> CALL_TOOL
  Step 5: +300 tool tokens = 1250 total -> BUDGET_EXHAUSTED (maxSteps=5 hit)
  Total time: ~20-25 seconds (5 LLM calls + 4 tool calls + infra overhead)

tool-heavy workflow (maxSteps=15, maxTokens=50000):
  Step 1: 200 input tokens -> CALL_TOOL
  Step 2: +300 tokens = 500 total -> CALL_TOOL
  ...
  Step 8: +300 tokens = 2300 total -> GENERATE_TEXT (40% probability each step)
  Total time: ~40-60 seconds (8 LLM calls + 7 tool calls + infra overhead)
```

Key observation: chat-light workflows terminate faster (fewer steps, tighter budget) but with a higher BUDGET_EXHAUSTED rate. tool-heavy workflows run longer and consume more resources, but are more likely to reach SUCCESS because they have room for more agent steps before the budget limit hits.

Phase 5: Contention Impact on Different Workloads

The multi-workload setup reveals how different request types affect each other:

- Chat-light workflows are fast individually but arrive 3x more frequently. They fill concurrency slots quickly, causing tool-heavy workflows to queue.
- Tool-heavy workflows hold concurrency slots much longer (more steps, more tool calls). A single tool-heavy workflow can block a slot for 40-60 seconds, during which 15-18 chat-light requests might have been served.
- Under high load, the queue becomes a mix of both types. A burst of chat-light arrivals can starve tool-heavy workflows, and vice versa.
- CPU contention affects both equally: all workflows share the same physical cores. The infrastructure layer does not distinguish between workload types.

Phase 6: Observing Results

The trajectory CSV and visualization scripts break down all metrics by workload. Key metrics to compare:

Metric	chat-light	tool-heavy
-----	-----	-----
Avg latency	~5-15s	~30-60s
Success rate	~40-60%	~60-80%
Budget exhausted	~40-60%	~10-20%
Avg steps	~3-5	~5-10
Avg cost	~\$0.002	~\$0.010
Queue wait time	lower (fast in)	higher (queued behind chat-light)

These numbers vary with configuration. The key insight is that you can now study how different workload mixes interact: changing chat-light's arrival rate from 3.0 to 0.3 would dramatically reduce queuing for tool-heavy workflows. This is the kind of capacity planning question the multi-workload simulator is designed to answer.

Timeline Visualization

The cumulative completions chart (Figure 3a in visualize_agentic.py) shows the two workloads progressing at different rates. chat-light completes faster and shows a steeper curve early on, while tool-heavy completions ramp up more gradually. The queue depth chart (Figure 4a) shows when infrastructure becomes saturated and workflows begin waiting for capacity.

21. Key Design Decisions

Why Discrete Event Simulation?

DES is the gold standard for modeling queuing systems. It is:

- Exact: no approximation errors from time-stepping
- Fast: jumps between events, doesn't tick through empty time
- Deterministic: same seed = same results = reproducible experiments
- Scalable: can simulate thousands of workflows in seconds

Why Gamma for LLM Latency?

Real LLM inference latency measurements show:

1. Always positive (no negative latency)
2. Right-skewed (most calls are fast, some are slow)
3. The tail varies by model/provider

Gamma distribution matches all three properties. The shape parameter controls the tail: low shape = heavy tail (more outliers), high shape = Gaussian-like.

Why Token-Based (Not Byte-Based)?

In agentic systems, the fundamental unit is the TOKEN:

- LLM latency scales with output tokens
- Cost is per-token
- Context window limits are in tokens
- Tool results are measured in tokens

v1 used bytes (for network bandwidth simulation). v2 uses tokens throughout.

Why Sealed Events?

Java sealed interfaces guarantee exhaustive matching. If you add a new event type, the compiler forces you to handle it everywhere. This prevents bugs where a new event type silently falls through.

Why Pluggable Decision Policy?

The LLMEngine takes a BiFunction for decisions, not a hardcoded strategy. This lets you:

- Use weighted random for statistical experiments
- Use scripted sequences for debugging (e.g., 'always call tool twice then finish')
- Swap in ML-based policies in the future
- Test edge cases (e.g., 'always fail')

Why Two Layers? (v1 + v2 Integration)

The simulator uses a two-layer architecture:

- Application layer (v2): models what the agent DOES (LLM calls, tool calls, decisions)
- Infrastructure layer (v1): models what the agent RUNS ON (CPU, pods, contention)

This separation exists because:

1. LLM inference is an external API call - latency depends on the model provider, not the local hardware.
2. But the agent's local work (context building, prompt formatting, response parsing) runs on physical CPUs and IS affected by hardware contention.
3. Under light load, the infra cost is negligible (~2ms). Under heavy load with many concurrent workflows sharing the same node, it can grow to 20ms+ and queue waits add even more.
4. There is NO network cost between the layers - they run on the same machine. The agent doesn't 'call' the infrastructure over a network.

The infrastructure layer is a live, stateful service. Each call sees the current system state (active jobs, queue sizes) and returns an estimate that reflects real contention at that moment.

Concurrency = Queuing Theory

The maxConcurrency + queue model is a direct M/M/c queue:

- M: Poisson arrivals (Markovian)
- M: Exponential service times (approximately)
- c: number of servers (maxConcurrency)

This lets you study queue buildup, tail latency under load, and optimal concurrency settings for different arrival rates.

22. Visualization

The project includes two visualization scripts for analyzing simulation output:

V1 Visualization: `visualize2.py`

Reads `simulation_trace_k8s.csv` and `queue_trace.csv` from the v1 Kubernetes simulation. Produces:

- Gantt chart showing request execution and wait times
- Queue depth per pod over time

```
python visualize2.py
```

V2 Visualization: `visualize_agentic.py`

Reads `agentic_trajectory.csv` from the v2 agentic simulation. Produces five figure groups, all broken down by workload type when multiple workloads are configured:

- Latency distribution: histogram and boxplot per workload
- Cost & steps: per-workflow cost over time, step count distribution
- Timeline & outcomes: cumulative completions, completion reason breakdown
- Infrastructure queuing: agent queue depth over time, wait time distribution
- Infrastructure latency: per-step infra latency, compute vs queue wait breakdown

```
python visualize_agentic.py
```

The workload column in the trajectory CSV enables per-workload filtering. When only one workload is defined, charts display aggregate data without workload-specific coloring.

V2 Trace Visualization: `visualize_trace.py`

A Gantt-style trace visualization that shows individual workflows flowing through agents in real time. This is the most detailed view, showing exactly which workflows overlap on each agent and what phase they are in at any moment.

- Agent Concurrency View: one panel per agent, workflows stacked vertically. Color-coded bars show queue wait (red), infra compute (orange), LLM inference (blue), and tool calls (green). Overlapping bars = concurrent execution.
- Concurrent Active Workflows: line chart showing how many workflows each agent is processing simultaneously over time, with per-agent and total lines.
- Combined Agent View: zoomed timeline showing workflows from all agents on one chart, grouped by agent with background shading for easy visual separation.

```
python visualize_trace.py          # default: 30 workflows
python visualize_trace.py --workflows 50  # show more workflows
python visualize_trace.py --start 0 --end 15000 # zoom to first 15s
```

This visualization is particularly useful for understanding multi-agent behavior: you can see agent-chat and agent-tool processing their respective workloads in parallel, each with independent concurrency pools but sharing the same CPU node.